

Sidekick predictive maintenance proposal

Theme 1 Agentic Predictive Maintenance Studio

Sidekick helps engineers find out whether an equipment-failure model still gives useful warnings when its sensors become unreliable. It also tests whether retraining can reduce that weakness.

1 Team Details

TEAM

Team Sidekick: Kareem Massoud and Adam Qablawi

2 Problem Statement and Relevance

CONTEXT

A failing machine can look healthy to a prediction model if an important sensor freezes at a normal reading. An engineer reviewing only the original test score may never see that weakness. The consequences could be a missed inspection or an unexpected shutdown.

Checking this properly takes more than training a classifier. Someone must prepare the sensor history, define what counts as a useful warning, reproduce sensor faults, and compare the resulting alerts. Sidekick brings that work into one application for maintenance and reliability engineers.

The intended benefit is a better model-selection decision: an engineer can see which faults affect the warnings, whether a proposed fix helps, and what remains uncertain before arranging a plant trial.

3 Idea and Approach

SOLUTION

The engineer uploads sensor histories and describes the maintenance question. Sidekick checks the data, trains a small set of candidates, and tests the same faults against each one. If a model depends too heavily on a sensor, it evaluates a version trained with corrupted readings as a possible remedy.

A replay view shows the original and altered sensor streams beside the model alerts. The engineer can ask, "Which model still warns early enough when this sensor freezes?" The assistant retrieves the relevant runs and explains the tradeoff. Every reported measurement links back to a stored result.

Three views in the prototype

View	What the engineer can do
Data setup	Confirm column mappings, inspect missing or constant readings, and set the prediction horizon and alert requirements.
Model comparison	Compare clean and faulted performance, inspect uncertainty, and open the results behind a recommendation.
Warning replay	Select an engine and fault, follow its alerts over time, inspect the signals behind a prediction, and export the selected model.

The proposal covers an offline prototype, followed by a possible monitored trial on plant data. Its contribution is a practical way to investigate and address model weaknesses before maintenance teams rely on the alerts.

Data and prediction target

The first benchmark is NASA C-MAPSS FD001, which provides 100 complete simulated turbofan-engine histories in its training file [1]. We will use these histories to predict whether failure is within the next 30 operating cycles. Remaining life is the final cycle minus the current cycle; values from 1 to 30 receive the positive label. The failure cycle is excluded.

The upload contract requires an equipment ID, an ordered cycle index, numeric sensor columns, and a known failure cycle for each complete history. The profiler checks duplicate IDs and cycles, ordering, gaps, missing values, constant channels, and label coverage. Ambiguous mappings require the engineer to confirm them. Censored histories need a separate protocol and are outside this first building.

Training and Comparison

Reserve 20 engines for one final evaluation. Use grouped five-fold cross-validation across the other 80, keeping each engine wholly within a fold. These are our own partitions of the complete training file. Fit every preprocessing step on the training fold; build features from the current reading and the preceding 19 cycles. The initial 19 cycles form a warm-up period.

Compare logistic regression, XGBoost, and an XGBoost variant trained with sensor faults. Add an age-based baseline that alerts once the cycle count exceeds a tuned threshold. Sensor models use reading and trailing means and slopes; engine ID, cycle count, and remaining life are excluded so that age dependence is explicit in the separate baseline.

Limit the search to three configurations per model variant. Use out-of-fold predictions to choose configurations and alert thresholds, then refit on all 80 development engines. Freeze that choice and the test plan before opening the final 20. The official truncated test set is reserved for later work.

Fault tests

Fault	Persistent test	Additional setting
Dropout	Remove one sensor stream from onset through the end of the recorded history.	Also test a 10-cycle outage.
Stuck reading	Repeat the reading immediately before onset through the end of the history.	Also test a 10-cycle freeze.
Drift	Ramp an offset for 20 cycles, then retain it. Test both signs.	Offset reaches 1 or 2 training standard deviations.

Apply faults separately to sensors that vary in the training data. Start the main tests 60 and 30 cycles before failure so they cover the warning period. These deliberately timed stress tests use known outcomes only to place faults, never as prediction inputs. A separate seeded random onset passes checks sensitivity to timing. Report the two sets separately; neither estimates how often faults occur in a plant.

Testing a remedy

The augmented variant mixes original training histories with corrupted copies. Fault positions and severities are sampled within each training fold using separate seeds. Labels remain unchanged. Missingness flags accompany imputed values; missing values use training media. Corruption happens before feature calculation. Compare the retrained version on both original and altered histories, including the stronger drift setting, and retain it only if the measured tradeoff meets the engineer's criteria.

Alert scoring and model selection

The demo assumes a minimum useful warning of 10 cycles and a prediction horizon of 30. These are benchmark settings to be agreed with an engineer for a plant trial. An alert starts after two consecutive scores reach the threshold and clears after two below it.

Measure	Definition
Useful detection	An alert is active at any point from 30 to 10 cycles before failure. Report on the number of detected engines and their fraction of the total.
Late or missed warning	No alert overlaps that window. Distinguish an alert appearing only in the last 9 cycles from no subsequent warning.
Early alarm burden	Fraction of eligible cycles with more than 45 cycles remaining spent in alarm, plus new alarm episodes per 1,000 such cycles.
Warning lead time	Cycles from the start of the first alert episode overlapping the useful window to failure. Report on the media alongside missed and late counts.

Cycles 31 to 45 form a transition band excluded from early alarm penalties. An earlier alert that remains active can therefore count as a detection, while time spent in alarm before that band remains visible. A continuously active alarm incurs 100% early alarm burden.

Before model selection, the engineer sets minimum useful detection and maximum early alarm burden. Show which candidates meet both on clean data and each required persistent-fault case. Rank qualifying candidates by mean detection across those cases, weighting cases equally. Display per-case counts and 95% Wilson intervals for engine detection; repeated fault runs do not count as additional engines. Report uncertain comparisons and allow no candidate to qualify.

5 Implementation and Impact

DELIVERY

Proposed responsibilities: Kareem leads data preparation and model evaluation; Adam leads the application and copilot. Both review scoring and integrate the demo. Development follows the 18 to 28 September prototype window.

Dates	Model evaluation	Application and delivery
18 to 20 Sep	Data checks, partitions, baseline models.	Upload flow, column mapping, job status, first tool calls.
21 to 23 Sep	Persistent faults, augmentation, scoring.	Comparison view, warning replay, linked run evidence.
24 to 25 Sep	Cross-validation, thresholds, configuration freeze.	Report checks, endpoint export, integration.
26 to 27 Sep	One final evaluation and reproducibility checks.	Demo recording, documentation, submission package.
28 Sep	Contingency for defects and final checks.	Delivery buffer; no new features.

Completion means a reproducible comparison, a working replay, an evidence-linked report, and a Docker-packaged prediction endpoint. Scripted checks cover ambiguous columns, missing labels, a single-engine upload, failed training, and a case where no model meets the requirements. A ranking change or successful retraining must come from the experiment, not a scripted result.

After the prototype, practitioner sessions will test whether Sidekick shortens model review and helps users recognize limitations. Compare the assistant with the same workflow operated through forms. Plant trials would then assess whether better model selection changes inspection decisions and downtime.

Python services own the calculations and recorded results. The copilot helps the engineer configure the work and investigate what comes back.

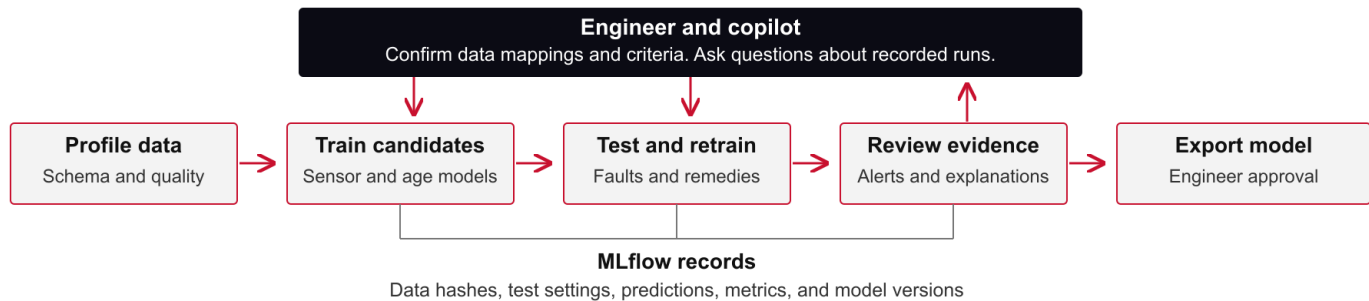


Figure 1 Proposed workflow and service responsibilities

Copilot behavior

GPT-5.4 mini will use function calling [2] to propose CSV mappings, translate alert requirements into explicit settings, and query MLflow runs. The engineer confirms mappings and criteria before training. Tools include `profile_dataset`, `map_columns`, `train_candidates`, `run_fault_tests`, `query_runs`, and `draft_report`.

For example, the engineer can ask, "Did retraining help under dropout without increasing early alarms?" The copilot retrieves paired runs, checks their data and test settings match, and explains the results with run links. Further diagnostics use development data. Final test results and acceptance criteria remain fixed.

Explanations and confidence

For an inspected tree-model alert, SHAP will show which inputs influenced the prediction. The comparison will include a reliability plot and Brier score from out-of-fold predictions, repeated on the final test set. This checks how predicted risks correspond to observed labels. Poorly calibrated estimates will be identified, and the LLM will not assign its own confidence to a numerical result.

Technical components

Component	Prototype choice
Interface	React for upload, comparison, and replay. FastAPI exposes jobs, results, and predictions.
Models and tests	scikit-learn, XGBoost, Pandas, NumPy, and SHAP. Fixed test functions rebuild features after sensor corruption.
Records	MLflow stores data hashes, splits, seeds, configurations, predictions, metrics, and model artifacts.
Copilot and export	Schema-validated API tools and a bounded Python workflow. Docker exports the chosen model with its preprocessing and report.

Raw sensor files stay in the backend; the LLM receives summaries and retrieved results. Tool arguments are validated, jobs have resource limits, and numerical statements are checked against stored metrics. Export requires an engineer's action. Prediction serving does not depend on the LLM.

Limitations and next steps

This is an idea-phase plan, with no measured performance or customer results yet. FD001 covers one simulated operating condition; it does not validate ABB motors or pumps. Twenty final-test engines limit precision. Fault severity needs field input, SHAP does not establish physical causes, and calibration can deteriorate under faults. Historian connections and production monitoring follow evaluation of the offline workflow and a plant-specific pilot.

Sources

[1] [NASA C-MAPSS dataset](#) FD001 data description. The classification task and evaluation partitions above are our proposed adaptation.

[2] [OpenAI GPT-5.4 Mini documentation](#) Function calling and structured outputs. Sources checked 13 September 2026.